Elastic Beanstalk

Class no:25

Elastic Beanstalk

Think of AWS Elastic Beanstalk as the "easy button" for deploying web applications. If you're a developer who wants to get your code onto the internet without spending hours configuring servers, load balancers, and scaling rules, this is your go-to service.

AWS Elastic Beanstalk: Overview

1. Definition

AWS Elastic Beanstalk is a **Platform as a Service (PaaS)** that allows for the quick deployment and management of applications in the AWS Cloud. You simply upload your code, and Elastic Beanstalk automatically handles the deployment details of capacity provisioning, load balancing, auto-scaling, and application health monitoring.

2. Key Components

- **Application:** A logical collection of Elastic Beanstalk components (environments, versions, and configurations).
- **Application Version:** A specific, labeled iteration of your deployable code (usually a Java .war file, a zip file, or a Docker image).
- **Environment:** A version that is deployed onto AWS resources.
    - *Web Server Environment:* Handles HTTP requests (standard web apps).
    - *Worker Environment:* Handles background tasks or long-running processes via an SQS queue.
- **Environment Tier:** Determines whether the environment runs a web app or a background task.

3. Supported Platforms

Elastic Beanstalk supports a wide variety of languages and platforms:

- **Programming Languages:** Java, .NET, PHP, Node.js, Python, Ruby, and Go.
- **Containers:** Docker.
- **Web Servers:** Passenger, Apache, Nginx, and IIS.

4. Why Use It? (Benefits)

- **Speed:** Go from code to a live URL in minutes.
- **Developer Productivity:** You manage the code; AWS manages the infrastructure.
- **Full Control:** While it automates the setup, you still have "root" access to the underlying AWS resources (like EC2 instances) if you need to tweak them.
- **Cost:** There is **no additional charge** for Elastic Beanstalk itself. You only pay for the underlying AWS resources (EC2, S3, etc.) that your application uses.

5. How It Works (The Workflow)

1. **Create Application:** Define your app name.
2. **Upload Version:** Bundle your code and upload it.
3. **Launch Environment:** AWS provisions the resources (EC2, Load Balancers, etc.).
4. **Manage:** Use the dashboard to monitor health, view logs, or update the application.

6. Deployment Strategies

- **All at Once:** Fast, but involves downtime.
- **Rolling:** Replaces instances in "batches" to avoid total downtime.
- **Immutable:** Creates a whole new set of instances to ensure the new version works before killing the old ones.
- **Blue/Green:** Swaps the CNAME of two separate environments for zero-downtime transitions.

Note: Elastic Beanstalk is perfect for startups and enterprise teams who want to stay focused on writing features rather than managing the "plumbing" of the cloud.

Fig.a. Components of Elastic Beanstalk

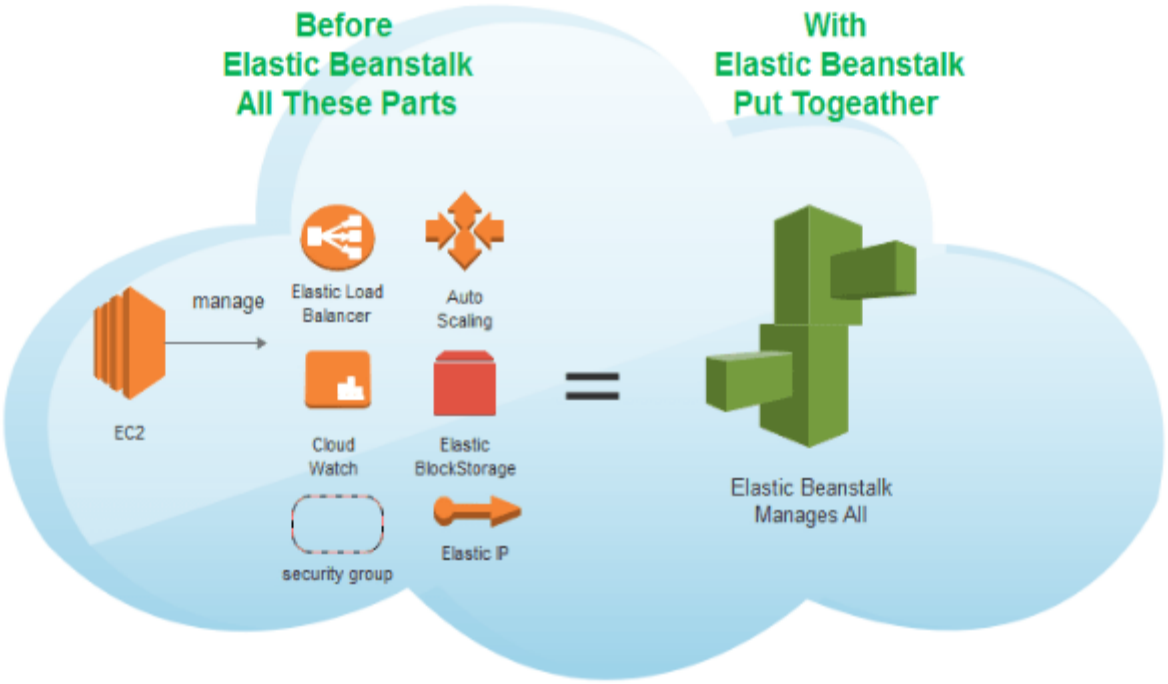
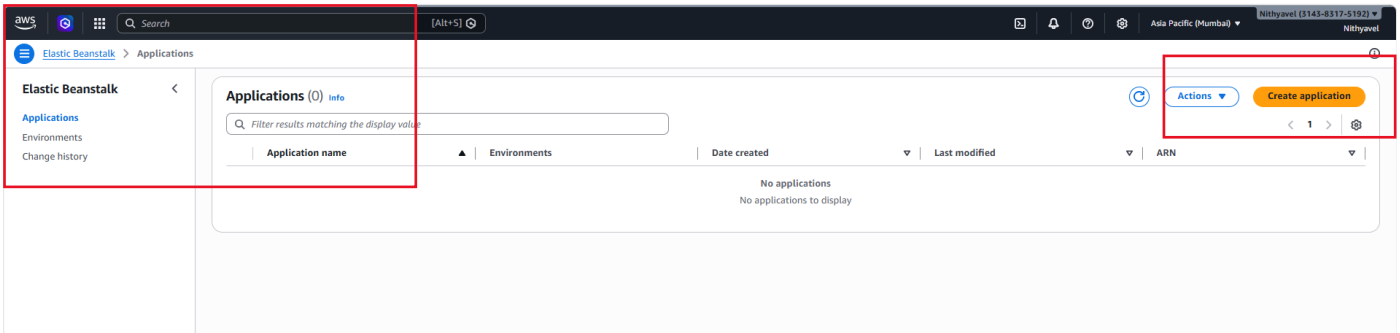


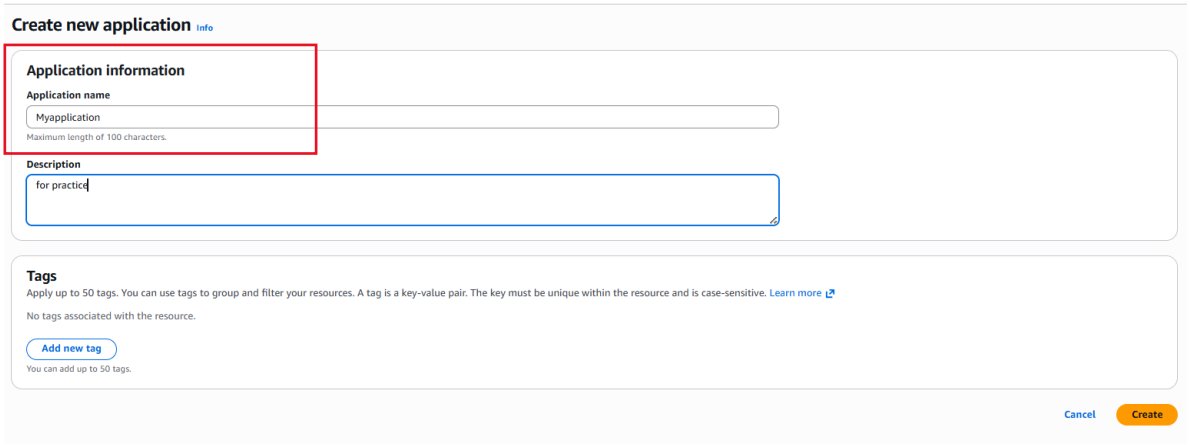
Fig.b.How Elastic BeanStalk works.

Hands on: How to Elastic Beanstalk and a EC2 instance for it.

Step no:1 Search **Elastic Beanstalk** → Application → Create Application.

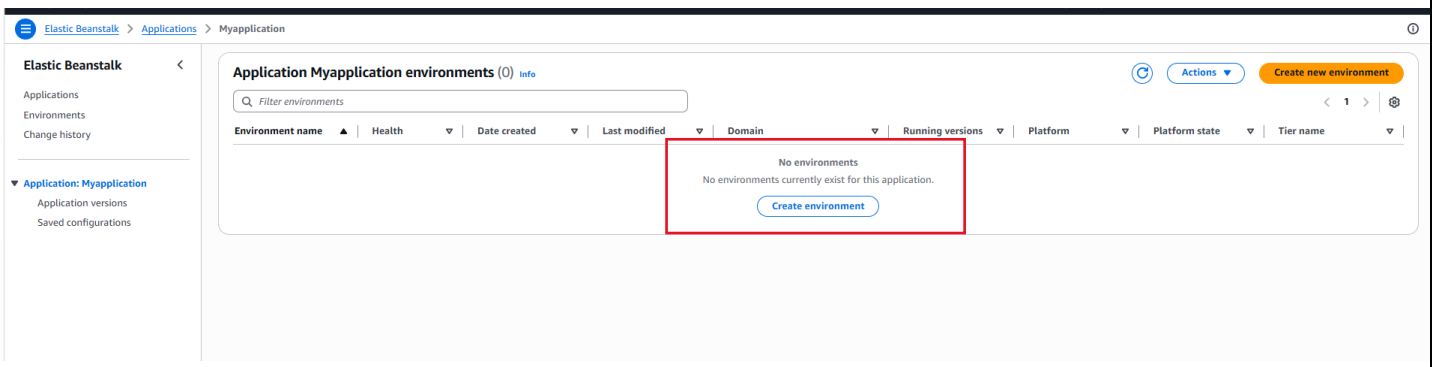


Step no:2 Provide a name: **Myapplication** → Create.



Step no:3 It will create an application. Now **Create new environment.**

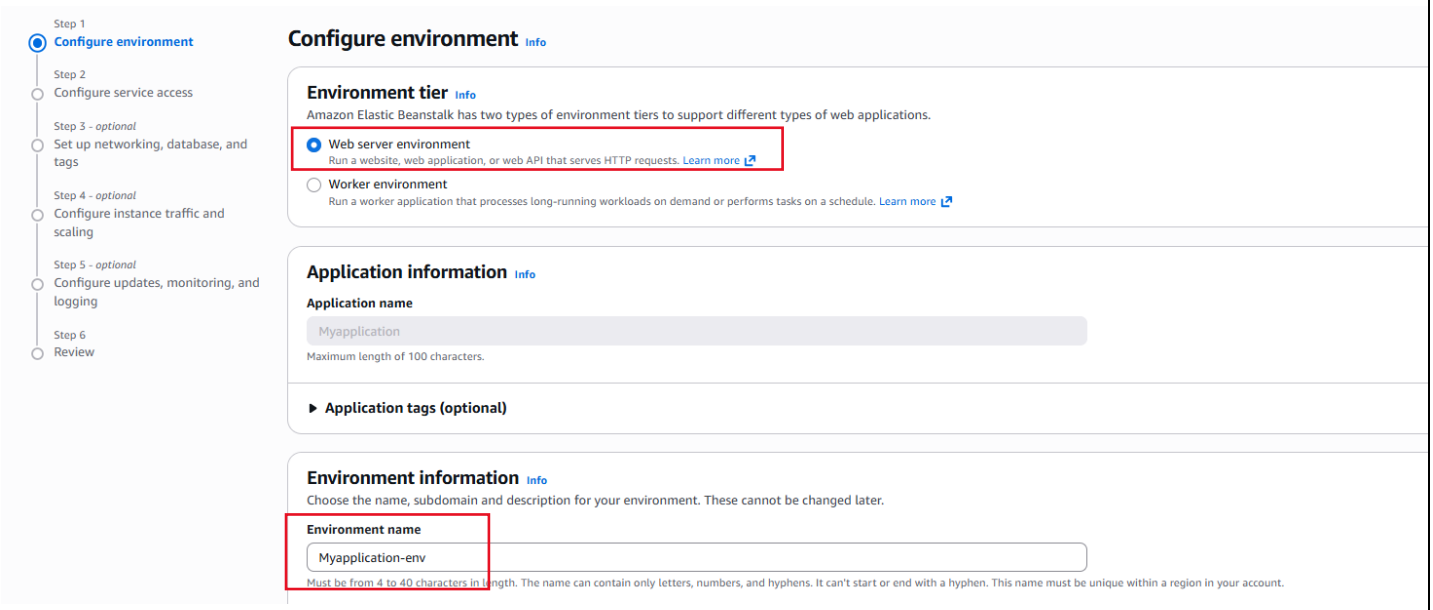
(We can create multiple environments for our application like development, staging and production environment).



Step no:4 Now configure the environment,

Tier: **Web server environment**

Environment name: **Myapplication-env**



Leave the domain name as blank, it will create a new one for us.

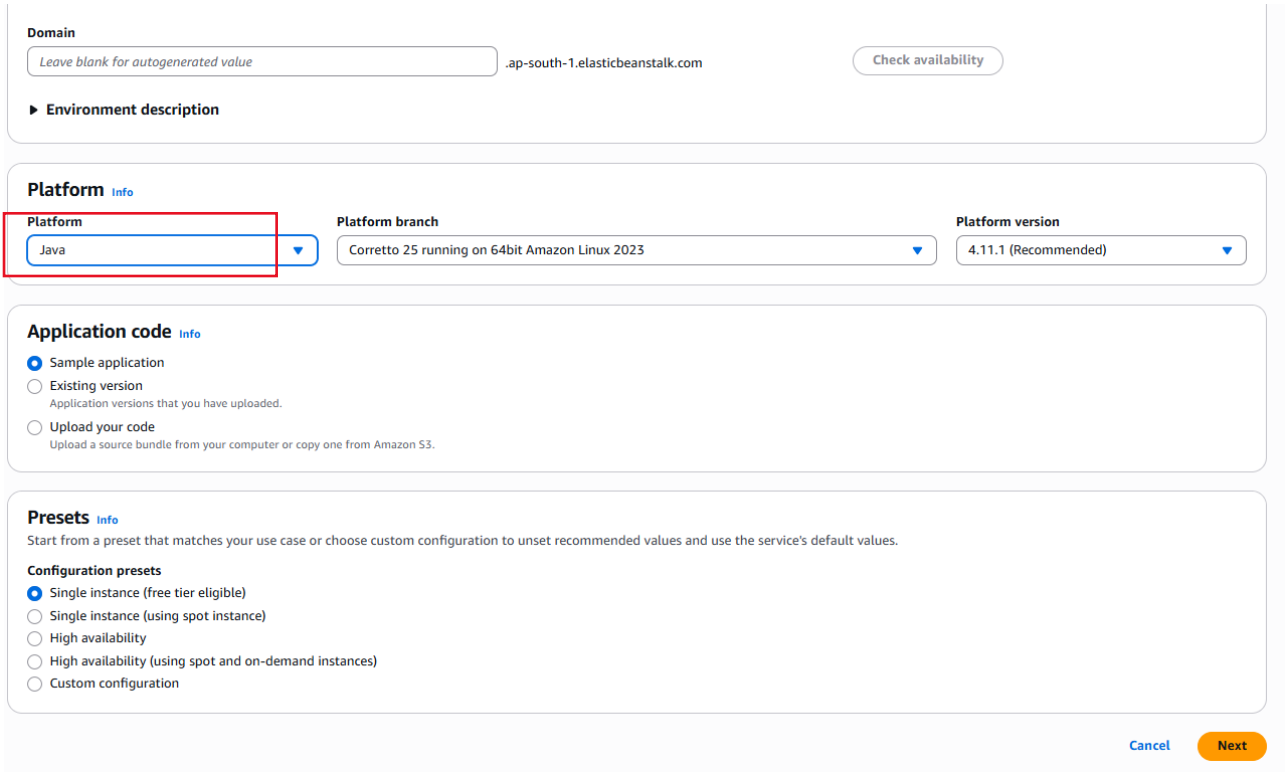
Platform: **Java**

Application code: **Sample application**

(We are just testing so use the existing code, there is also option to upload our code if we are deploying our own application)

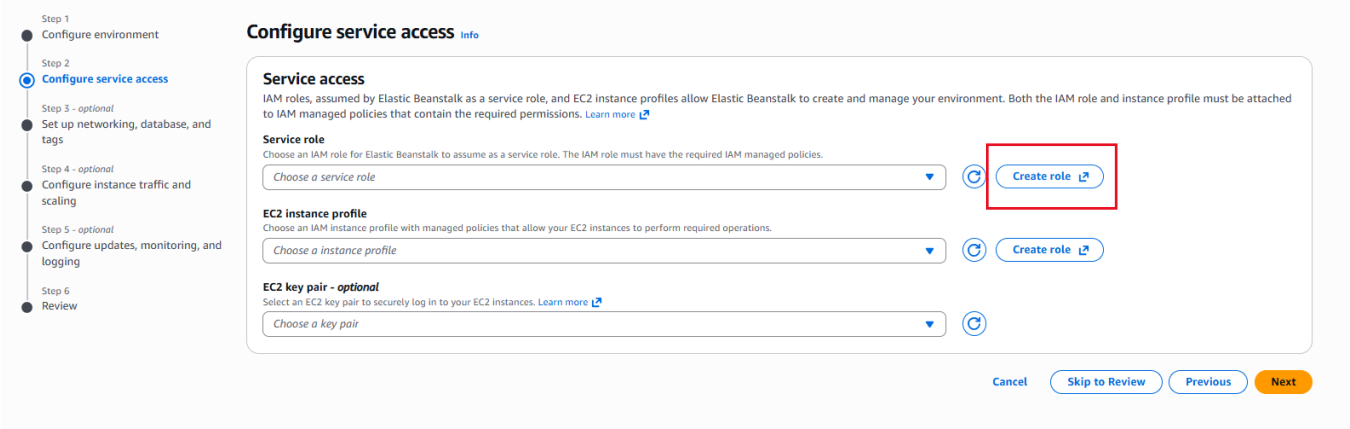
Presets: Choose the free tier eligible.

Next.

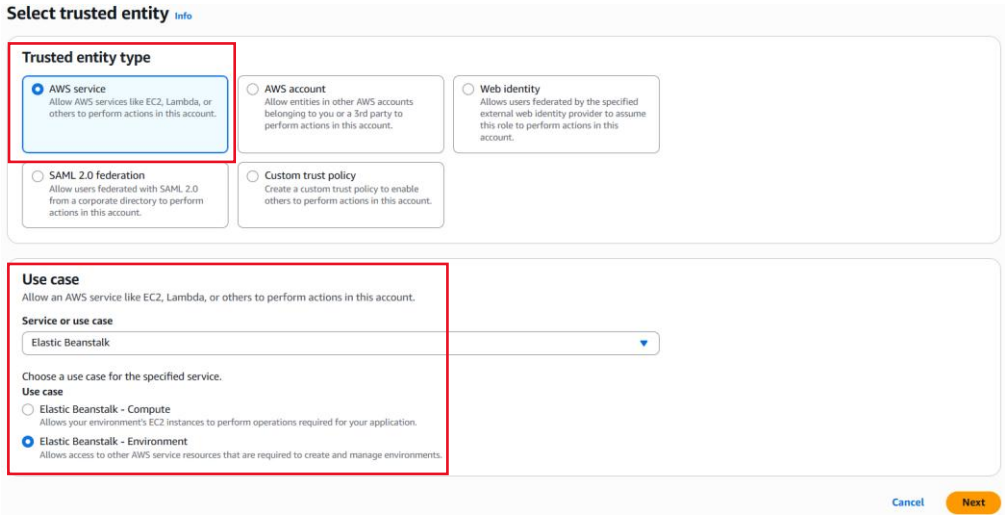


<u>Step no:5</u> For **service role** → click **Create role.** It will have a respective configured role.

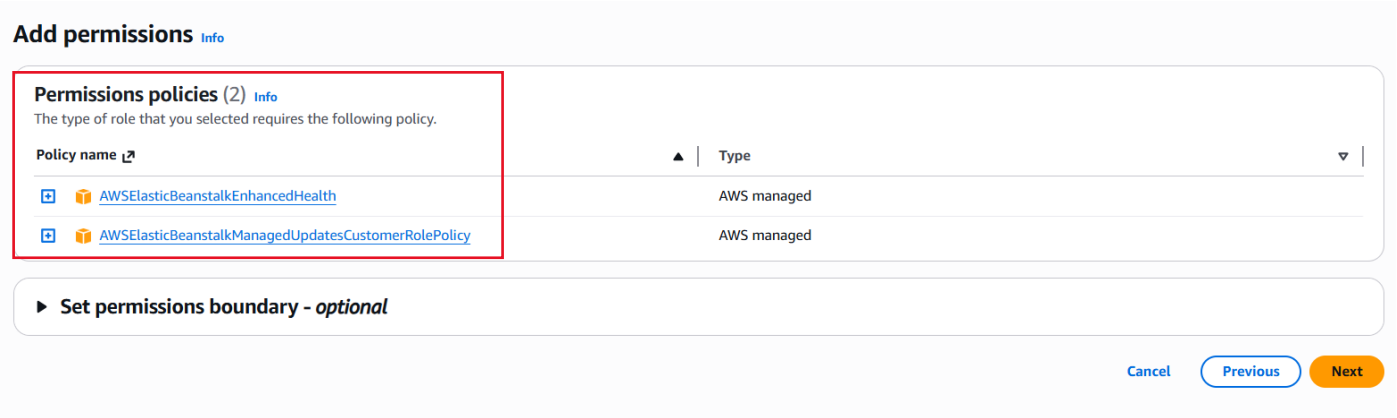
As we know in order to connect a service (EC2) with another service(EBS) we need a role (IAM).



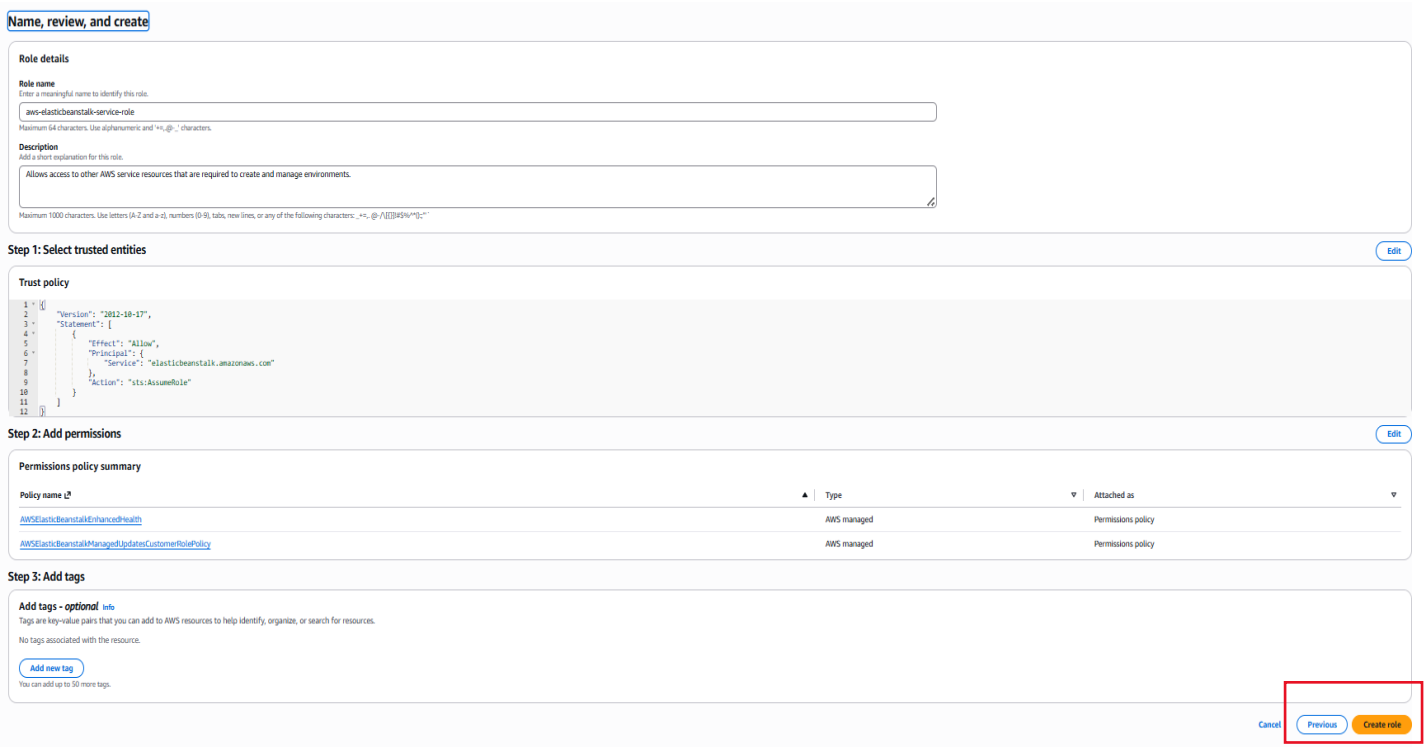
We can see the **service use case** as **Elastic beanstalk (env) → next.**



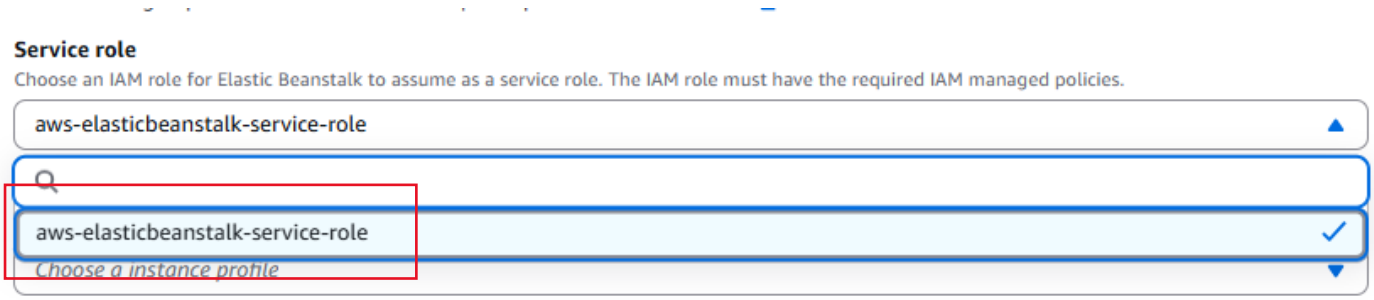
We can see it has automatically add the required permissions and policies → next.



Review → **Create role.**



Once created → do to our EBS page → **refresh** → we can see our own created role: **aws-elasticbeanstak-service-role.**



Similarly, **Ec2 instance profile.**

Choose a new / existing keypair.

Configure service access Info

Service access

IAM roles, assumed by Elastic Beanstalk as a service role, and EC2 instance profiles allow Elastic Beanstalk to create
to IAM managed policies that contain the required permissions. Learn more

Service role

Choose an IAM role for Elastic Beanstalk to assume as a service role. The IAM role must have the required IAM managed policies.

aws-elasticbeanstalk-service-role

EC2 instance profile

Choose an IAM instance profile with managed policies that allow your EC2 instances to perform required operations.

aws-elasticbeanstalk-ec2-role

EC2 key pair - *optional*

Select an EC2 key pair to securely log in to your EC2 instances. Learn more

mumkp

<u>Step no:6</u> Choose default **VPC,**

Enable Public IP

AZs: **Choose all for more availability**

Rest as default

Next.

Step 2
Configure service access

Step 3 - *optional*
Set up networking, database, and tags

Step 4 - *optional*
Configure instance traffic and scaling

Step 5 - *optional*
Configure updates, monitoring, and logging

Step 6
Review

Instance settings

Choose a subnet in each AZ for the instances that run your application. To avoid exposing your instances to the Internet, run your instances in private subnets and load balancer in public subnets. To run your load balancer and instances in the same public subnets, assign public IP addresses to the instances. Learn more

VPC

Launch your environment in a custom VPC instead of the default VPC. You can create a VPC and subnets in the VPC management console. Learn more

vpc-0538c231a226fe3c9 | (172.31.0.0/16) Create VPC

Public IP address

Assign a public IP address to the Amazon EC2 instances in your environment.

☑ Enable

Instance subnets

Filter instance subnets

Availability Zone	Subnet	CIDR	Name
ap-south-1a	subnet-01bf4d440c0912db3	172.31.32.0/20	
ap-south-1b	subnet-0b3c21b6de4314d95	172.31.0.0/20	
ap-south-1c	subnet-0c305b80f72541592	172.31.16.0/20	

Database Info

Integrate an RDS SQL database with your environment. Learn more

Enable database

Tags

Apply up to 50 tags. You can use tags to group and filter your resources. A tag is a key-value pair. The key must be unique within the resource and is case-sensitive. Learn more

Cancel | Skip to Review | Previous | Next

Choose the Security group as **default SG.**

Step 1
Configure environment
Step 2
Configure service access
Step 3 - *optional*
Set up networking, database, and tags
Step 4 - *optional*
Configure instance traffic and scaling
Step 5 - *optional*
Configure updates, monitoring, and logging
Step 6
Review

Configure instance traffic and scaling - *optional* Info

Instances Info

Configure the Amazon EC2 instances that run your application.

Root volume (boot device)

Root volume type	Size	IOPS	Throughput
The type of storage for the root volume attached to each instance.	The number of gigabytes of the root volume attached to each instance.	Input/output operations per second for a provisioned IOPS (SSD) volume.	The desired throughput for the Amazon EBS root volume attached to your environment's EC2 instance
(Container default)		100	125
	GB	IOPS	MiB/s

Amazon CloudWatch monitoring

The time interval between when metrics are reported from the EC2 instances

Monitoring interval

5 minute

Instance metadata service (IMDS)

Your environment's platform supports both IMDSv1 and IMDSv2. To enforce IMDSv2, deactivate IMDSv1. Learn more

IMDSv1

With the current setting, the environment enables only IMDSv2.

☑ Disable

EC2 security groups

Select security groups to control traffic.

EC2 security groups

Choose security groups Clear all

sg-06d80e515c32b9515 | default
default VPC security group

Environment type: **Single instance**

Fleet composition: **On-demand instance**

Instance types: **t3.micro and t3.small** (choose the free tier available instance types)

Then only it can create an instance on demand. → Next.

Capacity Info

Configure the compute capacity of your environment and auto scaling settings to optimize the number of instances used.

Auto scaling group

Environment type

Select a single-instance or load-balanced environment. You can develop and test an application in a single-instance environment to save costs and then upgrade to a load-balanced environment when the application is ready for production. Learn more

Single instance

Fleet composition

Spot instances are launched at the lowest available price. Learn more

- (●) On-Demand instance
- ( ) Spot instance

Architecture

The processor architecture determines the instance types that are made available. You can't change this selection after you create the environment. Learn more

- (●) x86_64
  This architecture uses x86 processors and is compatible with most third-party tools and libraries.
- ( ) arm64
  This architecture uses AWS Graviton processors. You might have to recompile some third-party tools and libraries.

Instance types

Add instance types for your environment with your preferred launch order. The order preference only applies to On-Demand Instances and Spot Instances that use the capacity optimized prioritized allocation strategy. We recommend you include at least two instance types. Learn more

1. t3.micro
2. t3.small Remove

Add instance type

AMI ID

Elastic Beanstalk selects a default Amazon Machine Image (AMI) for your environment based on the Region, platform version, and processor architecture that you choose. Learn more

ami-058121d1c7fafc3b9

Cancel | Skip to Review | Previous | Next

Keep the log and monitoring as default → Next.

Amazon X-Ray

Amazon X-Ray is a service that collects data about the requests and responses that your application serves and receives. You can use the tools that X-Ray offers to view and filter the data that it provides to identify potential issues and optimization opportunities.

X-Ray daemon
service charges may apply.
☐ Enable

S3 log storage

Configure the instances in your environment to upload rotated logs to Amazon S3. Learn more

Rotate logs
standard S3 charges apply.
☐ Enable

Instance log streaming to CloudWatch logs

Configure the instances in your environment to stream logs to CloudWatch logs. You can set the retention to up to 10 years and configure Elastic Beanstalk to delete the logs when you terminate your environment. Learn more

Log streaming
standard CloudWatch charges apply.
☐ Enable

Environment properties

Use environment properties as either plain text environment variables or as references to secret ARN values that are stored in AWS Secrets Manager or AWS Systems Manager (SSM) Parameter Store. When using secrets, update your Elastic Beanstalk EC2 instance profile role with permissions to your secrets and parameters. Learn more

Source	Name	Value	
Plain text	GRADLE_HOME	/usr/local/gradle	Remove
Plain text	M2	/usr/local/apache-maven/bin	Remove
Plain text	M2_HOME	/usr/local/apache-maven	Remove

Add environment property

Cancel Previous Next

Review and click **create.**

Review Info

Step 1: Configure environment

Environment information

Environment tier
Web server environment

Application name
Myapplication

Environment name
Myapplication-env

Application code
Sample application

Platform
arn:aws:elasticbeanstalk:ap-south-1::platform/Corretto 25 running on 64bit Amazon Linux 2023/4.11.1

Step 2: Configure service access

Service access Info

Configure the service role and EC2 instance profile that Elastic Beanstalk uses to manage your environment. Choose an EC2 key pair to securely lo

Service role
arn:aws:iam::314383175192:role/aws-elasticbeanstalk-service-role

EC2 key pair
mumkp

EC2 ins
aws-ela

Step 3: Set up networking, database, and tags

Step 5: Configure updates, monitoring, and logging

Edit

Updates, monitoring, and logging Info

Define when and how Elastic Beanstalk deploys changes to your environment. Manage your application's monitoring and logging settings, instances, and other environment resources.

Monitoring

System enhanced	**Cloudwatch custom metrics - instance** —	**Cloudwatch custom metrics - environment** —
Log streaming Disabled	**Retention** 7	**Lifecycle** false

Updates

Managed updates Enabled	**Deployment batch size** 100	**Deployment batch size type** Percentage
Command timeout 600	**Deployment policy** AllAtOnce	**Health threshold** Ok
Ignore health check false	**Instance replacement** false	

Platform software

Lifecycle false	**Log streaming** Disabled	**Logs retention** 7
Rotate logs Disabled	**Update level** minor	**X-Ray enabled** Disabled

Environment properties

Source	Key	Value
Plain text	GRADLE_HOME	/usr/local/gradle
Plain text	M2	/usr/local/apache-maven/bin
Plain text	M2_HOME	/usr/local/apache-maven

Cancel Previous Create

Step no:7 Once created check its **health** is **OK** → click it → we can see a **DNS** for the EBS.

Environments (1) Info

Filter environments by text, property or value

Environment name	Health	Application name	Platform	Domain
Myapplication-env	Ok	Myapplication	Corretto 2...	Myapplication-env.eba-nanafq...

Myapplication-env Info

Actions Upload and deploy

Environment overview

Health
Pending

Domain
Myapplication-env.eba-nanafqtb.ap-south-1.elasticbeanstalk.com

Environment ID
e-nq47pew5q5

Application name
Myapplication

Platform

Change version

Platform
Corretto 25 running on 64bit Amazon Linux 2023/4.11.1

Running version
–

Platform state
Supported

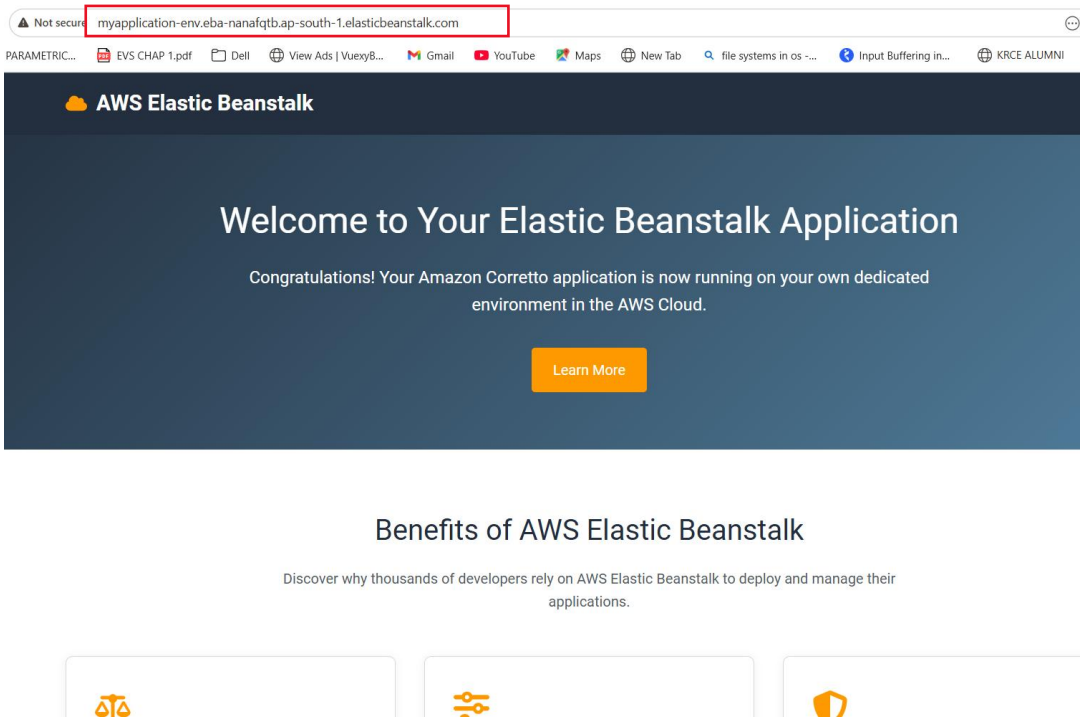
Events | Deployments | Health | Logs | Monitoring | Alarms | Managed updates | Tags

Events (10) Info

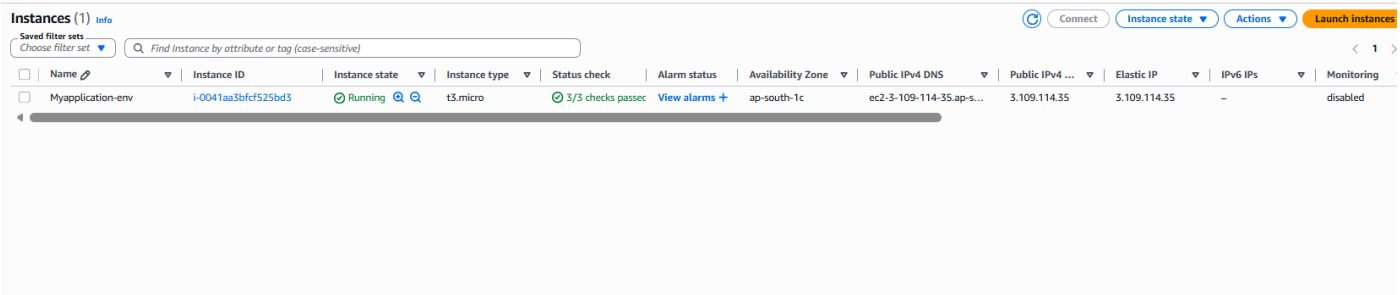
Filter events by text, property or value

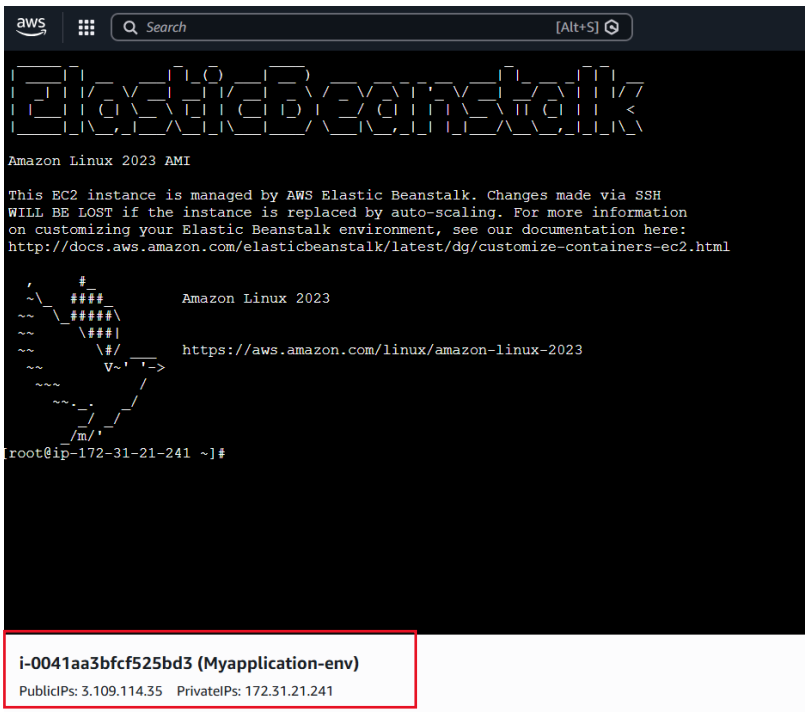
Time	Type	Details
April 15, 2026 17:45:27 (UTC+5:30)	INFO	Instance deployment completed successfully.
April 15, 2026 17:45:22 (UTC+5:30)	INFO	Added instance [i-0041aa3bfcf525bd3] to your environment.
April 15, 2026 17:45:20 (UTC+5:30)	INFO	Instance deployment used the commands in your 'Procfile' to initiate startup of your application.
April 15, 2026 17:45:20 (UTC+5:30)	INFO	Instance deployment successfully built your application using commands in the 'Buildfile'.

Copy and paste it in a private tab → we can see the webpage contents.



<u>Step no:8</u> Now go to **EC2** → We can see a **respective instance has been created** → named as **Myapplication-dev** (as provided by us in the EBS configuration). Select it and click **connect.**

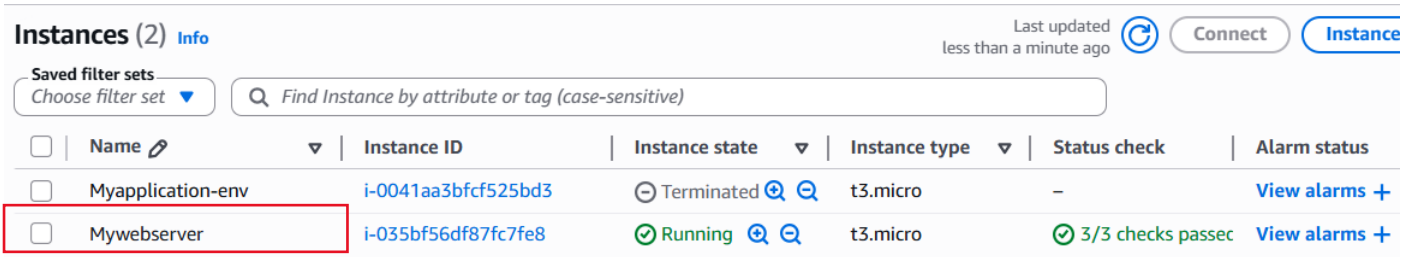




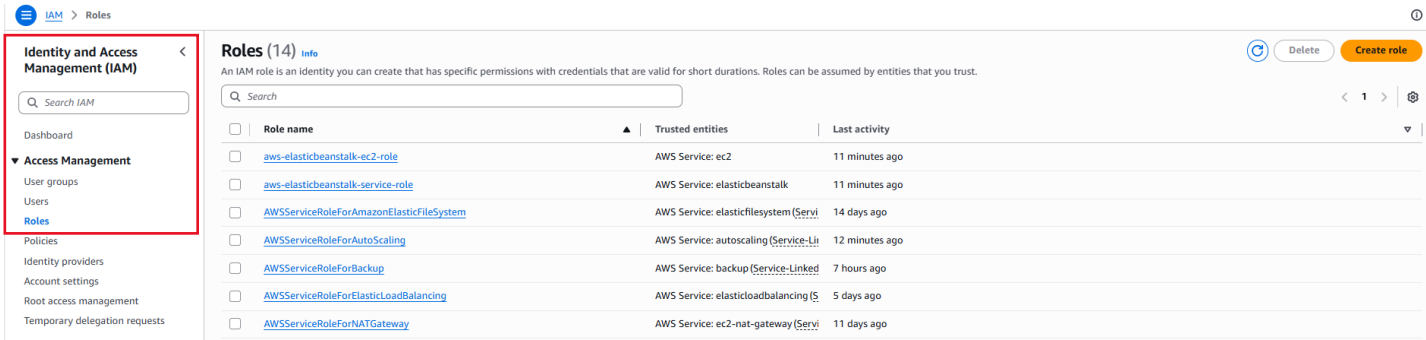
Task: To connect a EC2 instance via SSM Session Manager

Step no:1 First create a public instance.

Name: Mywebserever.



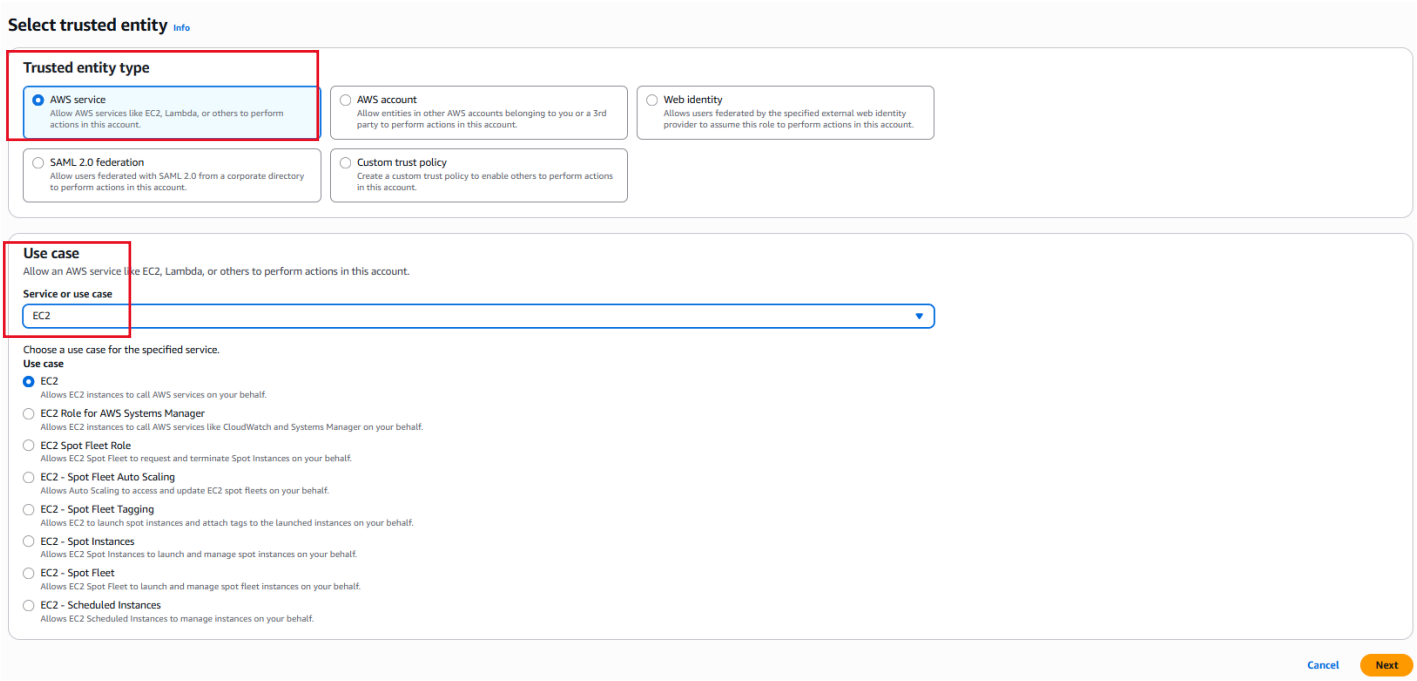
Step no:2 Once created Go to **IAM → Roles → Create role.**



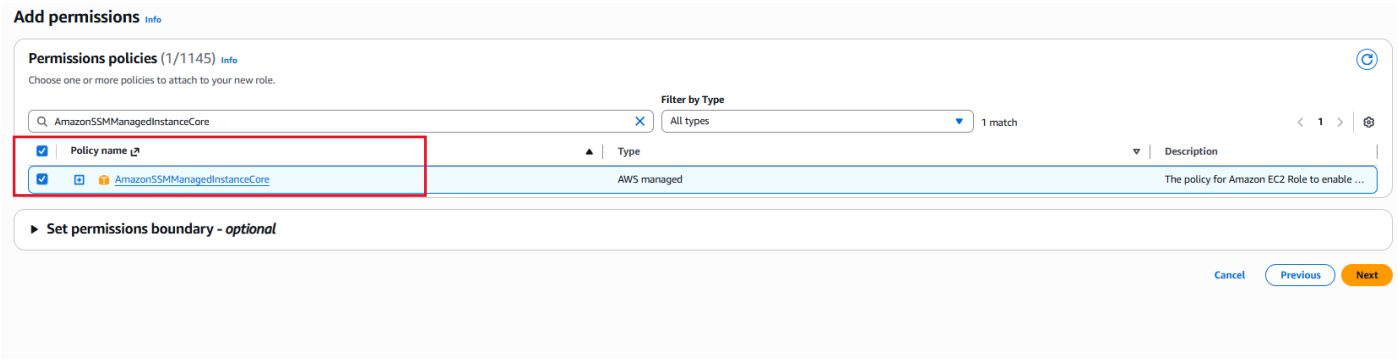
Entity type: **AWS service**

Use case: **EC2**

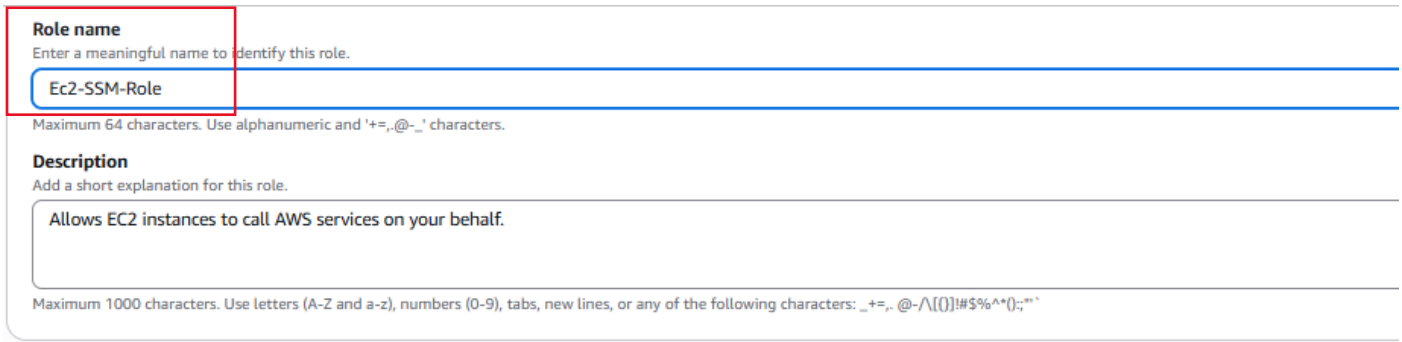
Next.

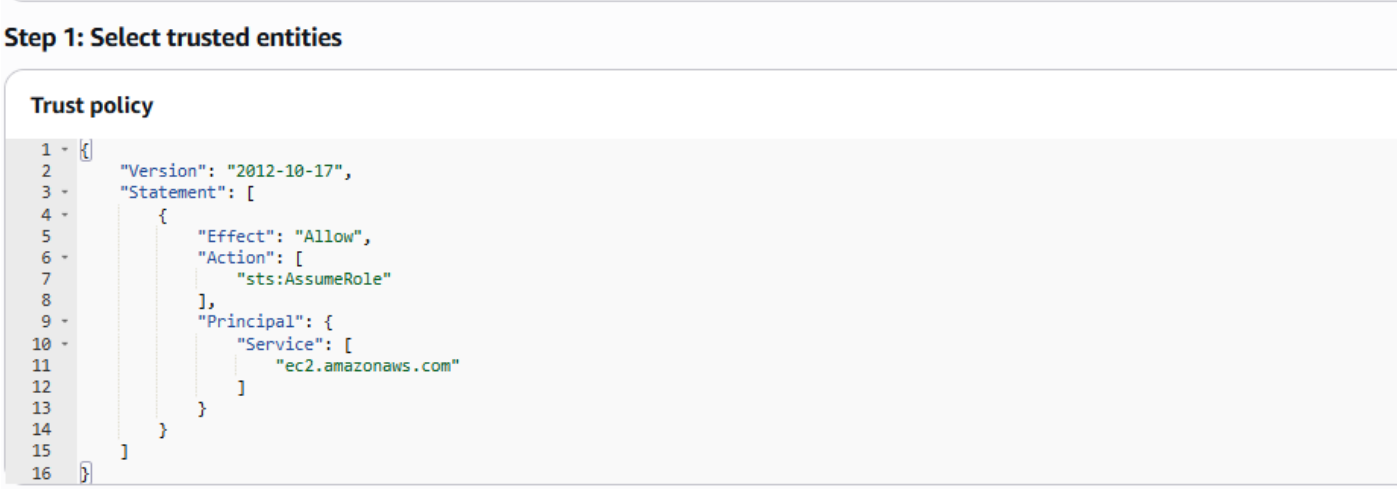


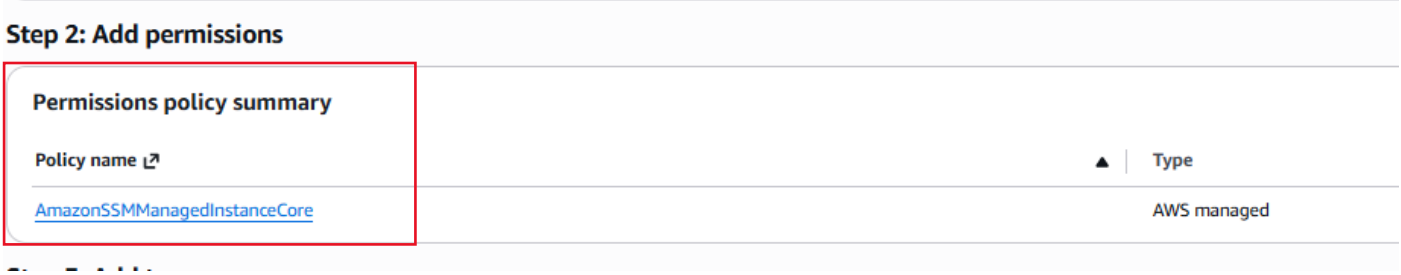
Permission → add permission → **AmazonSSMManagedInstanceCore** → next

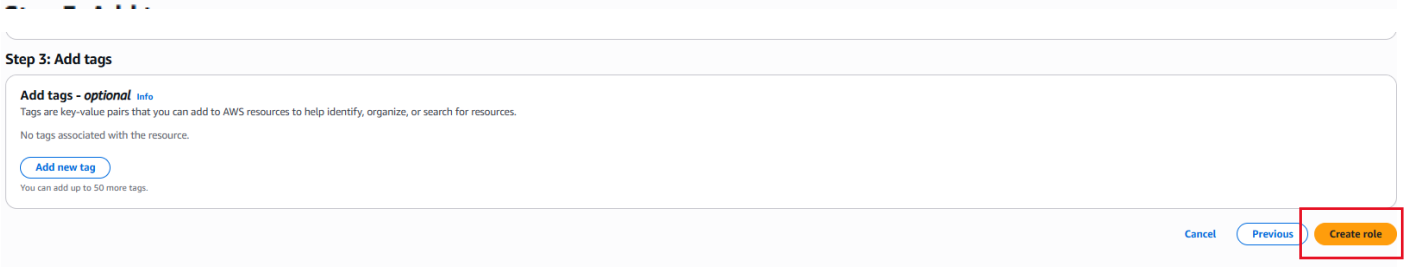


Provide a name for the policy: **Ec2-SSM-Role**



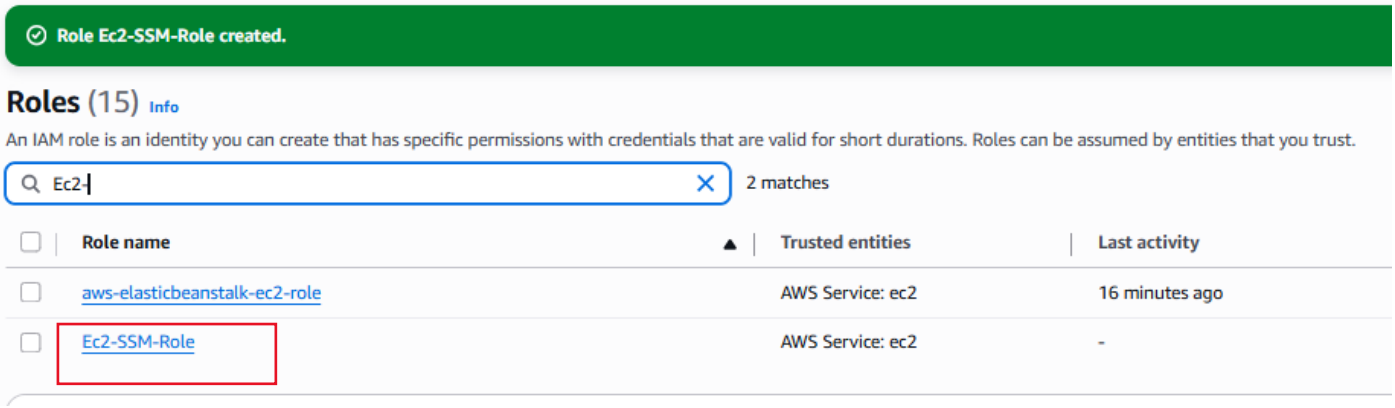




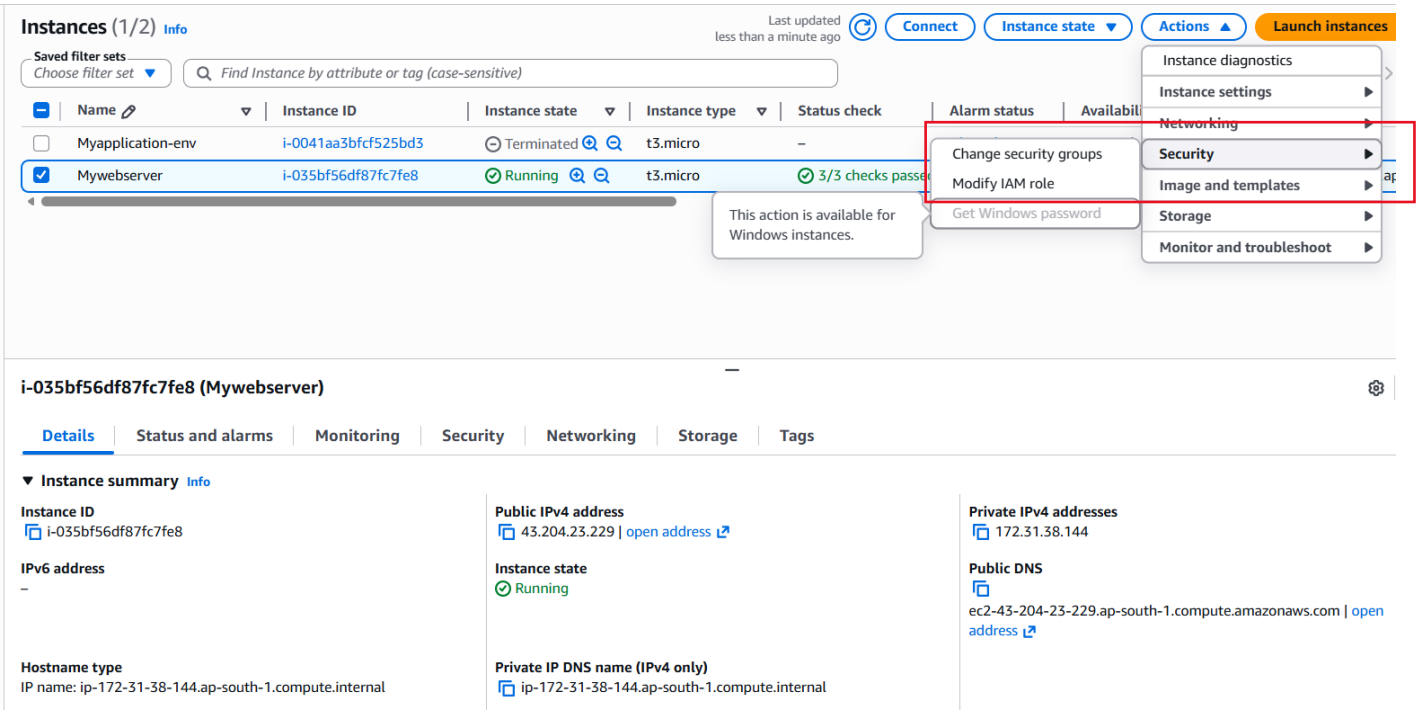


Review → **Create role.**

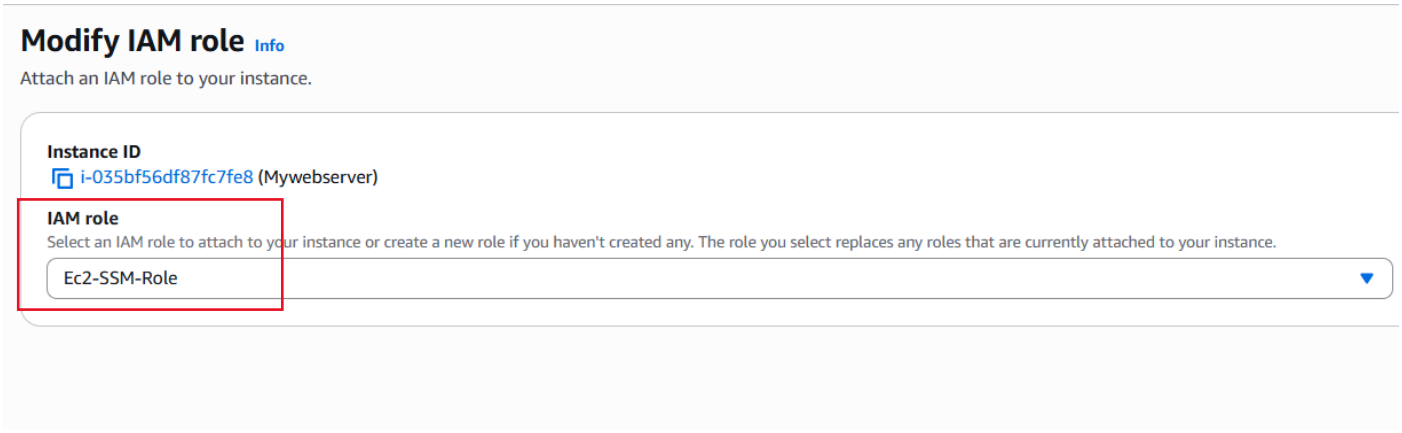
Once created we can see the role.



<u>Step no:3</u> Go our EC2 instance → **Mywebserver →Actions → Security → Modify IAM role.**

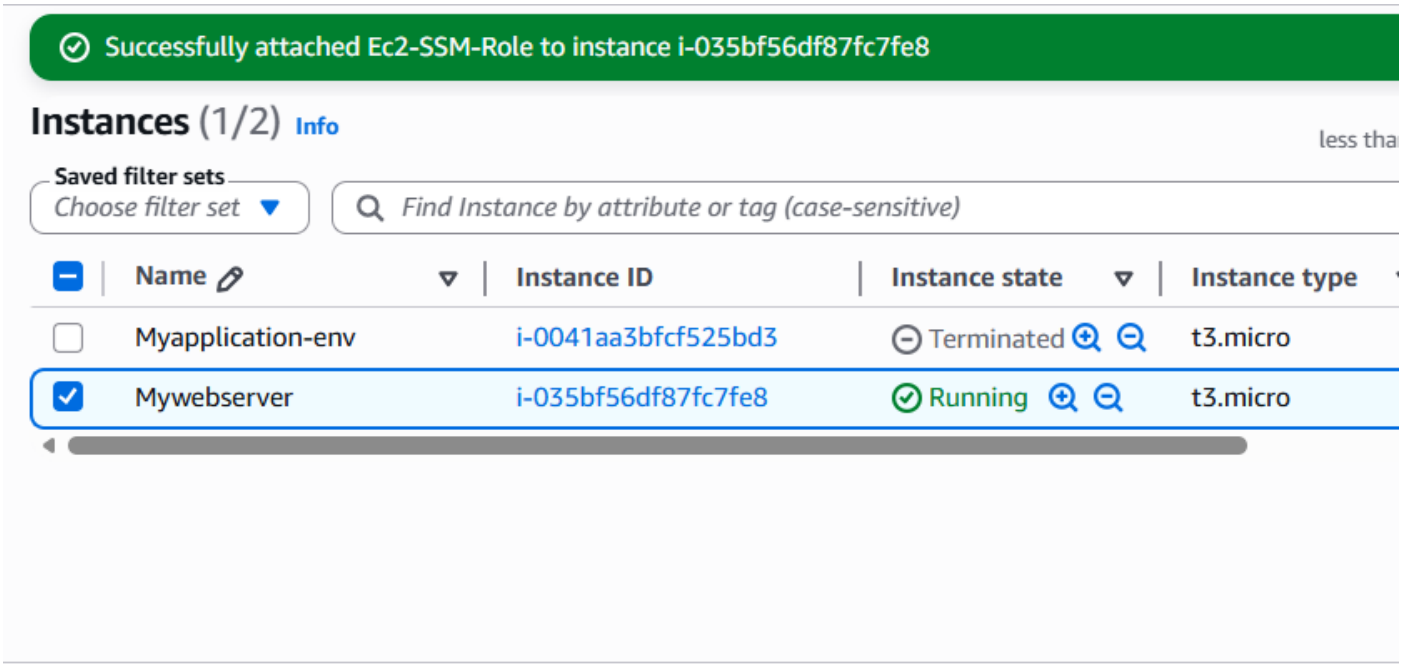


Choose the role we have created for SSM connect: **Ec2-SSM-Role.**



Once attached → **Reboot the instance.**

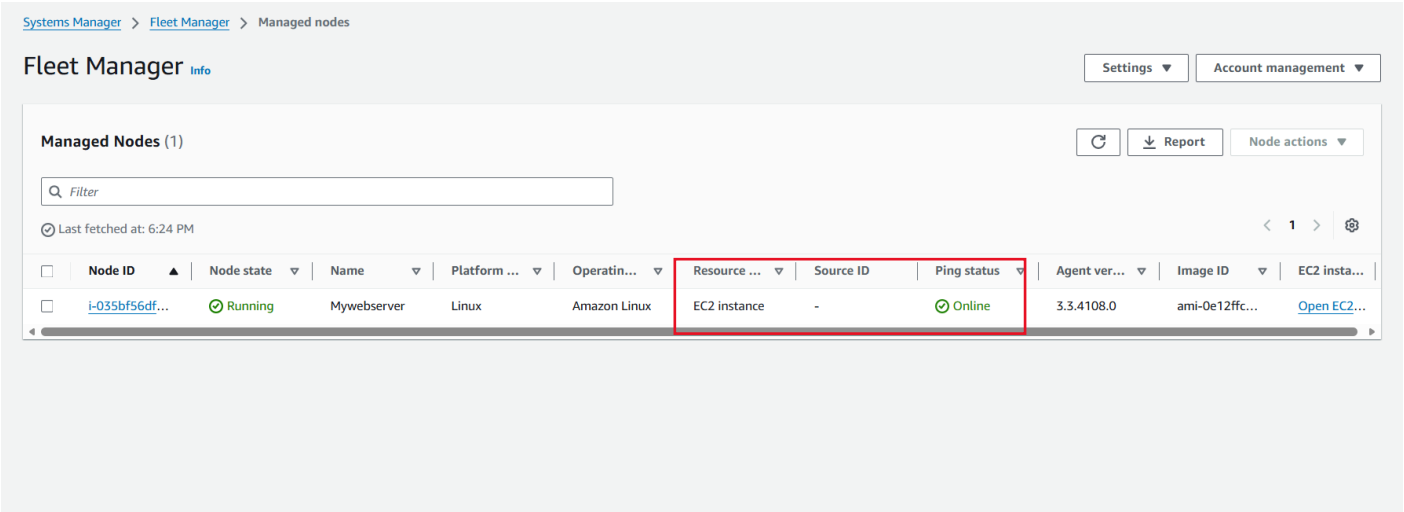
Then only the changes will be updated in the EC2.



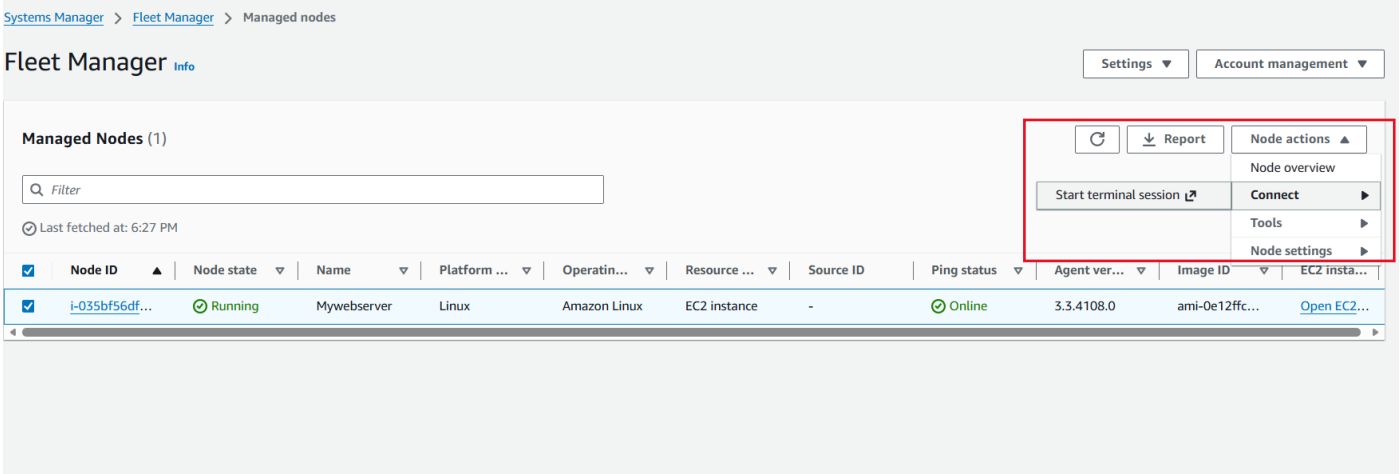
<u>Step no:4</u> Now search **System manager** in the console → click it → Under System manger scroll down → managed nodes → click **Fleet manager.**

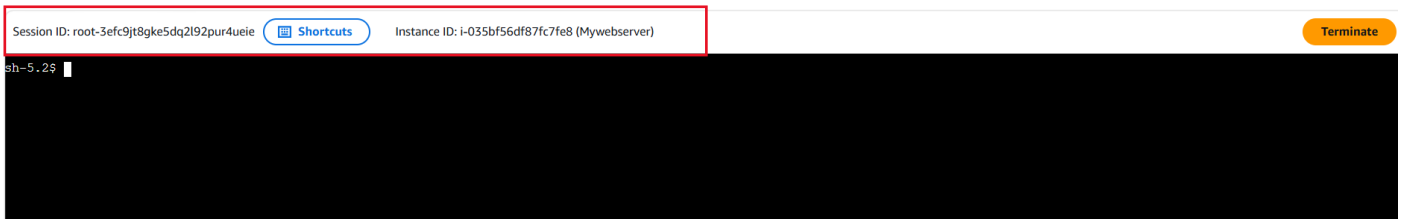
We can see our instance there. The status must be **online.** Then only it will work.

Also, once the IAM policy has been attached for the EC2 after rebooting wait for few minutes to get it reflected in the Fleet manager.

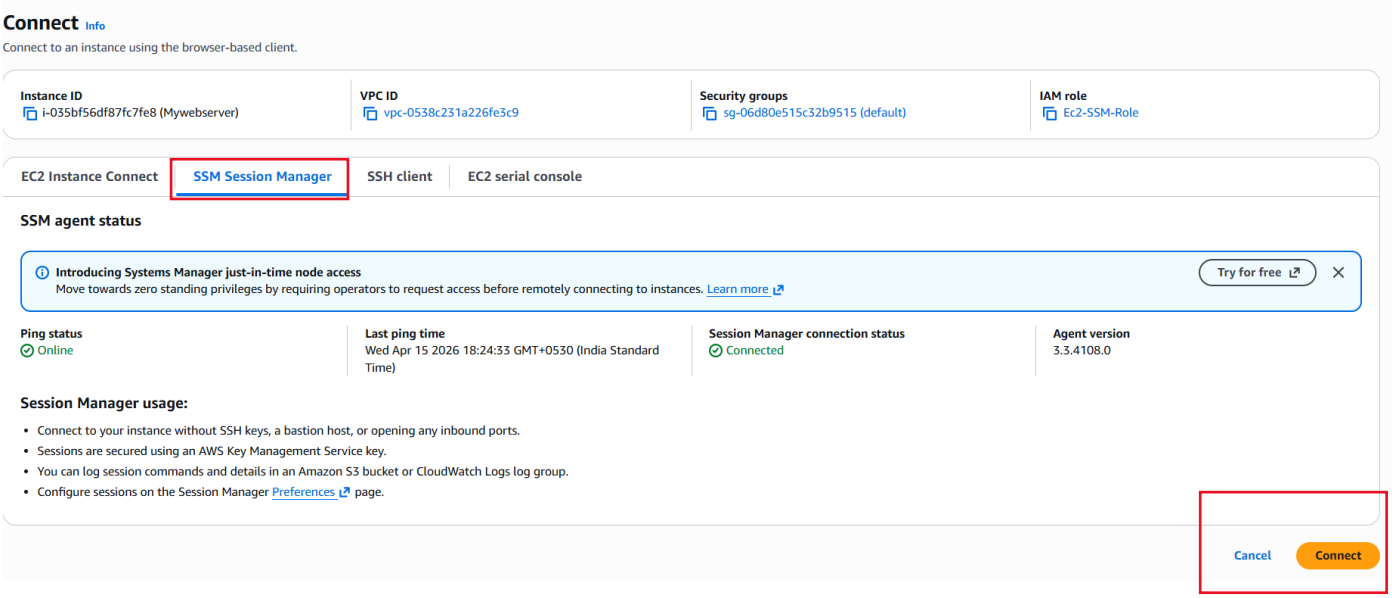


Select the node instance → **Node actions → connect → Start terminal session.**





Also we can connect from EC2 as well. Go to our **EC2 console** → Select our respective instance → click **connect** → **SSM Session Manager** → We can see the connect **enabled** → click **connect** → our instance will be connected.



Note: We are not logged in as EC2 user. Instead as a **SSM user**. We will have all sudo /admin access that can be performed.

To switch to home page as a EC2-user use the cd command: **cd /home/ec2-user.**